Retail SQL Agent — Architecture & Engineering Reference

A technical deep-dive for engineers ramping up on LLM-based agentic systems

Audience: experienced data/software engineers who know SQL and pipelines well, and are building a working mental model of agentic AI patterns.

Every claim in this document is traceable to a specific file and line in the repo — this is a reading companion, not a paraphrase.

1. Overview & Purpose

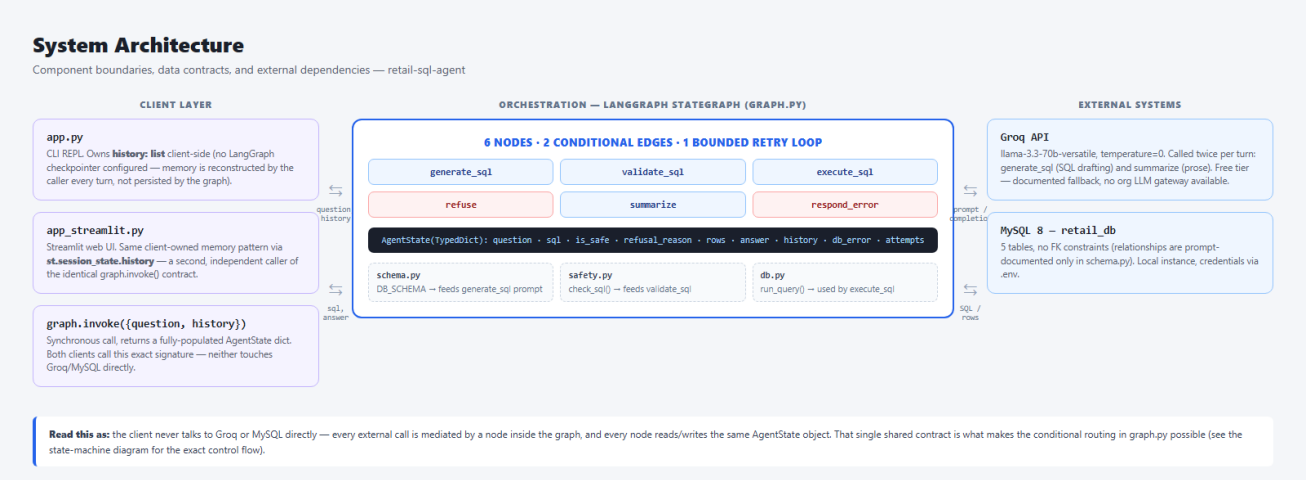
This repo answers natural-language retail business questions by generating read-only MySQL SQL via an LLM, validating it with a deterministic safety check, executing it, and returning a business-language summary. The interesting part isn't the SQL generation itself — it's the control structure wrapped around a single LLM call: validation, execution, error feedback, and a bounded retry loop, all modeled explicitly as a state machine rather than left implicit inside a prompt or a chain of `.then()` calls.

> **WHY AN EXPLICIT STATE MACHINE**
> A single mega-prompt ("write SQL, but only if it's safe, and retry if it fails") puts control flow inside the model's discretion — unobservable and unenforceable. LangGraph's `StateGraph` makes each decision point (is this safe? did it succeed? should we retry?) a real Python function that runs outside the model, with its own conditional edges. You can unit-test the routing logic independently of the LLM, which you cannot do with a single prompt.

This document walks the system top-down: architecture, then the exact state-machine wiring, then a file-by-file reference, then the two areas that matter most in any LLM-backed system — the safety gate and the prompts — followed by known issues, setup, and ideas for taking this from a learning exercise toward production.

2. System Architecture

Three layers: a thin CLI client, a LangGraph orchestration layer that owns all control flow, and two external systems (Groq for inference, MySQL for data) that are never called directly by the client — every external call is mediated by a node.



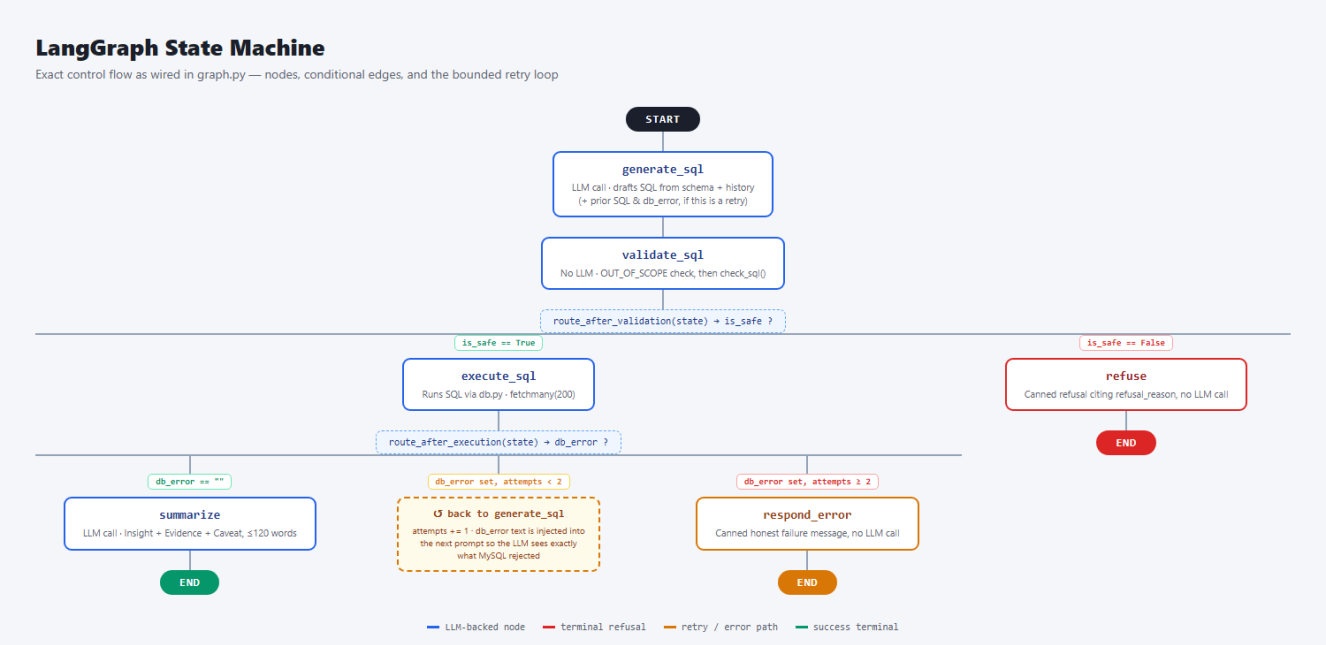
Component boundaries, the AgentState data contract, and external dependencies.

> **NO DURABLE SESSION MEMORY**
> There is no LangGraph checkpointer configured. `app.py` owns a plain Python `history` list and passes it into `graph.invoke({"question":..., "history":...})` on every turn, appending the result afterward. `app_streamlit.py` (Section 4) does the identical thing via `st.session_state.history` — a second, independent client on the same contract, not a change to it. Either way this is the simplest thing that works for a single session — it does not survive a process restart, and it doesn't generalize to multiple concurrent users. See Section 10 for what adding a checkpointer would buy you.

The single most load-bearing design decision in the architecture is that every node reads and writes the same `AgentState` object. That shared, typed contract is what makes the conditional routing in `graph.py` possible — a routing function is just a plain function of `AgentState` returning a string key.

3. The LangGraph State Machine

This is the exact control flow as wired in `src/agent/graph.py` — not a simplification. Six nodes, two conditional-edge functions, one bounded retry loop.



Nodes, conditional edges, and the retry loop, matching graph.py line for line.

3.1 The AgentState contract

Field	Type	Set by	Meaning
question	str	caller	the user's raw question
sql	str	generate_sql	the LLM's drafted query (or the literal string OUT_OF_SCOPE)
is_safe	bool	validate_sql	verdict from check_sql() / the OUT_OF_SCOPE check
refusal_reason	str	validate_sql	human-readable reason when is_safe is False
rows	list	execute_sql	dict rows from MySQL, capped at 200
answer	str	summarize / refuse / respond_error	the final text shown to the user
history	list	caller	prior {question, sql, answer} turns, client-owned
db_error	str	execute_sql	MySQL's exception text on failure, else ""
attempts	int	execute_sql	count of execute_sql failures so far

3.2 The two routing functions

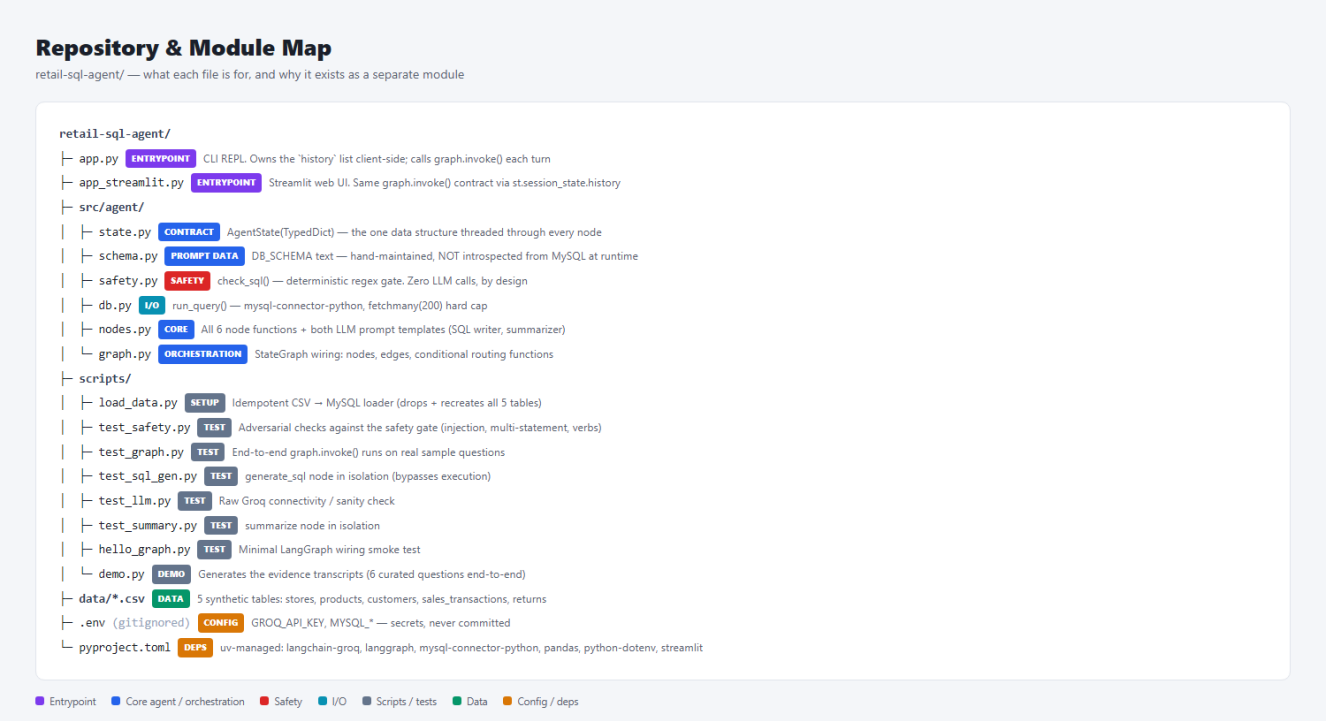
```
def route_after_validation(state: AgentState) -> str:
    return "execute_sql" if state["is_safe"] else "refuse"

def route_after_execution(state: AgentState) -> str:
    if not state["db_error"]:
        return "summarize"
    return "generate_sql" if state.get("attempts", 0) < 2 else "respond_error"
```

THE ATTEMPTS COUNTER IS EASY TO MISREAD

`attempts` is only ever incremented inside `execute_sql`'s `except` branch — success never touches it. Trace it through: 1st `execute_sql` failure sets `attempts=1`; `route_after_execution` sees `1 < 2` and retries. 2nd failure sets `attempts=2`; `2 < 2` is False, so it gives up. Net effect: at most 2 total `execute_sql` calls — one initial attempt plus exactly one regeneration retry, not two retries after the first attempt. Off-by-one misreads of this kind of counter are a common source of agent loops running longer (or shorter) than intended — always trace the exact increment site, not the variable name.

4. Module-by-Module Reference

The repo/file map below is the navigable version of what follows — use it to jump to the file you care about, then read the corresponding entry for the why, not just the what.



Every source file, its category, and a one-line purpose.

src/agent/state.py — the data contract

A single `TypedDict`. It exists as its own file, separate from `nodes.py`, purely so both `nodes.py` and `graph.py` can import it without a circular dependency — `graph.py` needs the type for `StateGraph(AgentState)`, and every node function needs it for its signature. TypedDict gives static-analysis benefits (IDE autocomplete, type-checker coverage on `state["..."]` access) without runtime validation overhead — LangGraph itself doesn't enforce the schema at runtime, so this is a documentation and tooling aid, not a guardrail.

src/agent/schema.py — the LLM's world model

A hand-written multi-line string (`DB_SCHEMA`) describing all 5 tables, their columns, and their join relationships, injected verbatim into every `generate_sql` prompt. It is not introspected from MySQL's `information_schema` at runtime — a deliberate simplicity tradeoff for a fixed, small schema.

> **MANUALLY-SYNCED SOURCE OF TRUTH**
> `scripts/load_data.py` has its own `SCHEMAS` dict that actually creates the MySQL tables. `schema.py`'s `DB_SCHEMA` string is a separate, hand-maintained description of the same tables for the prompt. Nothing keeps them in sync — if you add a column in one, you must remember to update the other by hand, or the LLM will confidently write SQL against columns that don't exist (or miss ones that do).

src/agent/safety.py — the deterministic gate

`check_sql(sql) -> (bool, str)`. Pure string/regex logic, zero LLM calls, zero network I/O. Three checks in order: reject if it contains a semicolon (multi-statement smuggling), reject if it doesn't start with `SELECT` (case-insensitive), reject if any of 12 forbidden keywords appear as a whole word (`\b`-bounded regex, case-insensitive). See Section 5 for the precision tradeoffs this makes.

src/agent/db.py — the execution boundary

One function, `run_query(sql) -> list[dict]`, wrapping `mysql-connector-python`. Notable details: `cursor(dictionary=True)` so rows come back as `{column: value}` dicts (JSON-friendly, easy to stringify for the summarizer prompt); `cursor.fetchmany(200)` as a hard cap regardless of how many rows actually matched, so a mis-scoped `SELECT *` can't flood the prompt context or the terminal; the connection is opened and closed per call inside a `try/finally` — no connection pooling, which is fine for a single-user CLI and a clear first thing to add for concurrent load.

src/agent/nodes.py — the six stations

All node functions and both prompt templates live in one file. Each node is a plain function taking `AgentState` and returning a partial dict — LangGraph merges the returned keys into the running state rather than requiring the full object back, so `execute_sql` only needs to return `{"rows":..., "db_error":...}`, not the whole state.

- generate_sql — builds the history block (last 3 turns, most-recent tagged explicitly), appends retry-error feedback if `db_error` is set, formats `SQL_WRITER_PROMPT`, calls the LLM, strips markdown code fences defensively.
- validate_sql — routes the `OUT_OF_SCOPE` sentinel straight to refusal before even calling `check_sql()`; otherwise delegates entirely to `safety.py`.
- execute_sql — the only node with a `try/except` around external I/O; on failure it captures `str(e)` verbatim as `db_error`, which is what makes the retry loop's error-feedback possible.
- summarize — truncates `rows` to the first 50 before stringifying them into the prompt (see the gotcha callout in Section 6) and formats `SUMMARY_PROMPT`.
- refuse / respond_error — pure string formatting, no LLM call, no I/O. These are the two "cheap failure" terminal nodes.

src/agent/graph.py — the wiring

Imports all six node functions and both routing functions, calls `StateGraph(AgentState)`, registers nodes with `add_node`, registers the two conditional forks with `add_conditional_edges`, and the three straight-line edges (`summarize`, `respond_error`, `refuse` all go to `END`) with `add_edge`. `builder.compile()` produces the importable `graph` object — this is the only place in the codebase where the shape of the pipeline is defined; every other file only defines behavior at a single node.

app.py — the CLI client

A `while True` REPL. The entire client-side contract with the graph is one call: `graph.invoke({"question": question, "history": history})`. It's worth noting this is a synchronous, blocking call — there's no streaming of intermediate node output, so the user sees nothing until the entire pipeline (up to 3 LLM calls in the worst case: 2 generate_sql attempts + 1 summarize) has finished.

app_streamlit.py — the web client

A second, independent client over the same `graph` object — `src/agent/` was not touched to add it. Streamlit reruns the entire script top-to-bottom on every user interaction, so state that must survive a rerun lives in `st.session_state` rather than local variables:

```
if "history" not in st.session_state:
    st.session_state.history = [] # exact shape app.py uses
if "display_log" not in st.session_state:
    st.session_state.display_log = [] # richer per-turn data, UI-only

result = graph.invoke({"question": question, "history": st.session_state.history})
```

> **TWO LISTS, ON PURPOSE**
> `st.session_state.history` stores only `{question, sql, answer}` — byte-identical to what `app.py` builds — because that's the object fed back into `generate_sql`'s prompt, and changing its shape would change agent behavior. A separate `display_log` list carries the extra fields the UI needs to render (`rows`, `refusal_reason`, `db_error`) that were never meant to reach the LLM. Conflating the two — e.g. appending the full `AgentState` result to `history` — would silently change what the model sees on every subsequent turn. Keep a renderable log and an LLM-context log separate whenever a UI needs to show more than the model needs to remember.

Rendering branches on `refusal_reason` vs. `db_error` vs. neither — `st.warning` for a Guard refusal, `st.error` for a DB failure that survived both retries, `st.success` otherwise — surfacing the same three terminal states the state machine defines (Section 3), rather than collapsing everything into one text field the way `app.py`'s `print()` does. The `graph.invoke()` call is wrapped in `try/except` so a runtime failure (e.g. MySQL unreachable) renders an error banner for that turn without crashing the session — an uncaught exception mid-script would otherwise kill that rerun and dump a traceback into the UI.

5. The Safety Model — Deep Dive

The prompt already instructs the LLM to only write SELECT statements. `safety.py` exists anyway, and is the actual enforcement point — this is deliberate defense-in-depth, not redundancy. An LLM instruction is a strong preference; `check_sql()` is a hard boundary that runs identically regardless of what the model decided to do.

```
FORBIDDEN = ["insert", "update", "delete", "drop", "alter", "create",
             "truncate", "replace", "grant", "revoke", "call", "load"]

def check_sql(sql):
    cleaned = sql.strip().rstrip(";").strip()
    if ";" in cleaned: # rule 1: single statement
        return False, "Multiple SQL statements are not allowed."
    if not cleaned.lower().startswith("select"): # rule 2: SELECT-only
        return False, "Only SELECT queries are allowed."
    for word in FORBIDDEN: # rule 3: keyword blocklist
        if re.search(rf"\b{word}\b", cleaned, re.IGNORECASE):
            return False, f"Forbidden keyword detected: {word.upper()}."
    return True, ""
```

KNOWN FALSE-POSITIVE SURFACES

This gate is intentionally biased toward false positives over false negatives, which is the right call for a safety boundary, but worth knowing precisely: (1) the semicolon check fires on any semicolon, including one inside a string literal value (`WHERE notes = 'Hi; there'`) — a legitimate query gets rejected. (2) the keyword scan is `\b`-bounded, so it correctly ignores `DELETE` as a substring of `deleted_at` (no boundary between "delete" and the following "d"), but it will reject a legitimate `SELECT` that merely references the word "grant" as data (e.g. `WHERE category = 'grant'`), because the blocklist has no concept of SQL syntax — it only sees tokens. Both are acceptable tradeoffs here, but a blocklist is fundamentally different from a parser, and it's worth being explicit with your team about which one you're actually running.

WHY THIS CAN'T BE AN LLM CHECK

A second LLM call asking "is this SQL safe?" would be non-deterministic, promptable-around, and unauditable — you can't write a unit test that proves an LLM will refuse a given input 100% of the time. `check_sql()` is 20 lines of pure Python; `scripts/test_safety.py` asserts against it directly with zero network calls, zero flakiness, and a result you can point to in a security review.

6. Prompt Engineering Notes

6.1 SQL_WRITER_PROMPT

Structure, in order: role framing ("expert MySQL analyst") → full `DB_SCHEMA` → conditional history block → a fixed rules list → the user's question. The rules list is where most of the iteration happened in practice (see Section 8):

- Output contract — "Return ONLY the SQL. No explanations, no markdown fences" — necessary because LLMs default to wrapping code in ``` fences; `generate_sql` also defensively strips them in code as a second layer.
- COUNT vs. SUM disambiguation — an explicit rule mapping "most/how many" to COUNT and "highest value/how much" to SUM, added after the model conflated the two (a real bug — see Section 8, catch #1).
- History recency tagging — the history block labels entries `[MOST RECENT]` vs. `[N turns ago]` rather than listing them undifferentiated, and the rules explicitly say a follow-up refers to the `[MOST RECENT]` turn (added after a real bug — Section 8, catch #5).
- The MySQL 1235 workaround — an explicit rule telling the model never to use `LIMIT` inside an `IN (subquery)` (unsupported by MySQL) and to use a derived-table JOIN instead, with a worked example inline in the prompt.
- OUT_OF_SCOPE sentinel — a magic string convention, not a boolean flag or structured output. `validate_sql` does an exact string match on it. This is a deliberately low-tech mechanism — works fine at this scale, would not survive contact with a model that phrases refusals inconsistently.

6.2 The retry-feedback block

When `state["db_error"]` is set, `generate_sql` appends a second block to the prompt containing the exact previous SQL and the exact MySQL exception text, plus a hint pointing at the derived-table JOIN pattern. This is the mechanism documented in Section 3 as the retry loop — the model isn't retrying blindly, it's being handed the specific failure to correct.

6.3 SUMMARY_PROMPT

Enforces a fixed 3-part structure (Insight / Evidence / Caveat), a 120-word cap, and an explicit "never invent numbers not in the rows" grounding instruction — the closest thing this project has to a hallucination guardrail, and it's a prompt instruction, not a verified constraint. Nothing programmatically checks that the numbers in `answer` actually appear in `rows`.

> **DOUBLE TRUNCATION**
> `execute_sql` caps at 200 rows via `fetchmany(200)`. `summarize` then takes only `rows[:50]` before stringifying them into the prompt. For aggregate questions (SUM/COUNT/AVG) this is invisible — you get one row back either way. For a row-listing question that legitimately returns more than 50 rows, the Narrator silently only ever sees the first 50, and its "Evidence" section is grounded in an unlabeled subset. Worth flagging to a user of this system, and worth fixing with an explicit "(showing 50 of N rows)" note if you extend this.

7. Data Model

Table	Rows	Key columns
stores	10	store_id (PK), store_name, city, region, open_date
products	50	product_id (PK), product_name, category, unit_price, unit_cost
customers	200	customer_id (PK), customer_name, city, segment, join_date
sales_transactions	5,000	transaction_id (PK), store_id, product_id, customer_id, quantity, total_amount, payment_method
`returns`	250	return_id (PK), transaction_id, return_date, refund_amount

NO REAL FOREIGN KEYS

MySQL has no FK constraints between these tables — all four relationships (`sales_transactions.store_id -> stores.store_id`, etc.) exist only as prose in `schema.py`. Joins work because the LLM reads and follows that prose correctly, not because the database enforces referential integrity. `returns` is also a MySQL reserved word and must stay backtick-quoted everywhere — the schema prompt calls this out explicitly because it's exactly the kind of thing a model gets wrong silently otherwise.

8. Known Issues Found During Development

Documented in full in `ai_usage.md`. Reproduced here with root cause / fix framing, because reading real agent bugs is the fastest way to build intuition for where these systems actually break.

Catch #1 — COUNT vs. SUM confusion

Symptom: asked "which store had the most returns," the agent returned the store with the highest refund value (Ahmedabad Express, ₹273,041, only 16 returns) instead of the highest count (Delhi Mall, 42 returns). Root cause: the prompt didn't disambiguate "most" (a count) from "highest" (a value). Fix: an explicit rule mapping the two phrasings to COUNT vs SUM. Verified: independent hand-written SQL confirmed the corrected output.

Catch #2 — history silently dropped

Symptom: follow-up questions had no memory of prior turns. Root cause: `history` was not declared as a field on the `AgentState` TypedDict, so it wasn't reliably threaded through. Fix: declared `history: list` on the state schema.

Catch #3 — MySQL error 1235

Symptom: the LLM repeatedly generated `LIMIT` inside `IN (subquery)` for "top N" filtering, which MySQL rejects outright (error 1235, "This version of MySQL doesn't yet support..."). Fix: a prompt rule forbidding the pattern with a derived-table JOIN example, plus the retry loop as a runtime safety net — this is the concrete case the retry loop was built for.

Catch #4 — digit-grouping slips

Symptom: summaries occasionally misplaced comma digit-groups when restating large INR figures in prose. Mitigation: all cited figures in evidence artifacts were cross-checked against raw MySQL output; no prompt fix fully eliminates this class of error since it's a text-generation artifact, not a logic bug — flagging it to end users as a caveat is the honest mitigation.

Catch #5 — follow-up misattribution with 3+ history turns

Symptom: with several unrelated questions ahead of it in history, a follow-up ("and only for the Mumbai store?") sometimes reused an earlier turn's query structure instead of the immediately preceding one. Root cause: the history block listed the last 3 turns with an undifferentiated "Previous question / Previous SQL" label for all of them — nothing told the model which one was most recent when more than one was structurally similar. Fix: explicit `[MOST RECENT]` / `[N turns ago]` tagging (Section 6.1) plus a prompt rule pointing follow-ups at the tagged turn. Verified: re-ran the same conversation; the corrected output matched independently hand-written SQL.

9. Setup & Running

Prerequisites: Python 3.12+, `uv`, MySQL 8 running locally with `retail_db` created. Package manager is `uv`; there is no separate lint/build step.

```
createdb / mysql -u root -p -e "CREATE DATABASE retail_db;"
cp .env.example .env # fill in GROQ_API_KEY + MYSQL_*
uv sync # install dependencies
uv run scripts/load_data.py # idempotent — drops + recreates all 5 tables
uv run app.py # interactive chat REPL (CLI)
uv run streamlit run app_streamlit.py # same agent, browser UI
```

Tests are plain standalone scripts, no pytest runner:

```
uv run scripts/test_safety.py # adversarial checks against the safety gate
uv run scripts/test_graph.py # end-to-end graph runs on sample questions
uv run scripts/test_sql_gen.py # generate_sql node in isolation
uv run scripts/test_llm.py # raw Groq connectivity check
uv run scripts/test_summary.py # summarize node in isolation
uv run scripts/demo.py # generates evidence transcripts in evidence/
```

10. Extending This Toward Production

This is a learning-exercise-scale system. If you were taking it further, roughly in priority order:

1. Durable session memory — add a LangGraph checkpointer (e.g. Postgres- or Redis-backed) instead of the client-owned `history` list, so conversations survive process restarts and can be resumed across requests.
2. Structured output / tool calling — instead of parsing raw SQL text out of an LLM completion, use a tool-calling schema so the model returns a structured query object, closing off an entire class of prompt-injection and markdown-fence-stripping fragility.
3. Stronger safety boundary — a real SQL parser (e.g. `sqlglot`) validating an AST-level allow-list instead of regex/keyword matching would close the false-positive/false-negative gaps described in Section 5.
4. Observability — trace every node invocation (prompt, completion, latency, token count) with LangSmith or OpenTelemetry; today the only visibility is print statements in `app.py`.
5. Eval harness — replace the manual smoke-test scripts with a golden question set asserting expected SQL shape and/or expected answer content, run in CI on every prompt change — prompt changes are code changes and should be regression-tested like any other.
6. Cost/rate tracking — Groq's free tier has no built-in cost visibility in this codebase; a production version needs per-request token/cost logging before scaling usage.
7. Schema sync — auto-generate `DB_SCHEMA` from `information_schema` (or at least from the `load_data.py` `SCHEMAS` dict) instead of hand-maintaining a second copy, closing the gap flagged in Section 4.

11. Agentic AI Concepts, for an Engineer New to Them

LLM	A model trained to predict text continuations, exposed here via Groq's API. Treat it as a stateless function: text in, text out, no persistent memory between calls — which is exactly why this project reconstructs conversation context by re-sending history on every call.
Temperature	A sampling parameter controlling output randomness. This project uses 0 (lowest) for both prompts, since query generation and structured summaries both benefit from determinism over creativity. Note: temperature=0 reduces but does not guarantee identical outputs across calls — Groq's inference stack is not perfectly deterministic in practice (observed empirically during testing, not a bug in this codebase).
Prompt engineering	Here it means: everything in `SQL_WRITER_PROMPT` and `SUMMARY_PROMPT` beyond the raw question — schema context, few-shot-style examples (the derived-table JOIN hint), explicit disambiguation rules, and output-format contracts. Treat prompt text as source code: it has bugs (Section 8), it needs review, and it should be under version control (it is — it's just Python string literals in `nodes.py`).
Agent vs. chain	A "chain" is a fixed sequence of calls. An "agent" implies some conditional/looping control flow driven by intermediate results — which is exactly what `route_after_validation` and `route_after_execution` provide here. This project is a small, fully-deterministic-control agent: the paths through the graph are fixed and inspectable, only the content of `sql` and `answer` comes from the model.
Grounding / hallucination	"Grounding" means constraining a model's output to verifiable source data. The summarizer is grounded by construction — it's only ever shown the actual `rows` from MySQL and instructed not to invent numbers — but that grounding is enforced by prompt instruction, not by a programmatic check (see Section 6.3). This is a real gap between what the prompt asks for and what the system actually guarantees.
Context window / history truncation	Only the last 3 conversation turns are included in the `generate_sql` prompt (`history[-3:]`), regardless of how long the conversation actually is — a deliberate bound on prompt size and cost, with the tradeoff that a reference to something more than 3 turns back will not resolve correctly.
Why no RAG here	Retrieval-Augmented Generation would search a corpus to find relevant context per query. This system doesn't need it: the entire "knowledge base" is a ~30-line schema description that fits comfortably in every prompt, so it's injected wholesale rather than retrieved selectively. RAG becomes relevant once the context you'd need no longer fits in a prompt budget — worth knowing the threshold, not just the pattern.
Deterministic vs. stochastic validation	The central architectural lesson of this codebase: `safety.py` is deterministic (same input, same output, always, provably) while both LLM calls are stochastic (same input, probably similar output, not provably anything). Put safety-critical decisions on the deterministic side of that line whenever you can.

12. Understanding Check

Quick self-check — if any of these don't feel obvious, that's a pointer back to the relevant section.

1. Why does safety.py exist at all, given the prompt already instructs SELECT-only output?

Answer: Defense-in-depth: an LLM instruction is a strong preference, not a guarantee. check_sql() is the actual, provable enforcement point — 20 lines of deterministic code you can unit-test with zero flakiness.

2. After exactly one execute_sql failure followed by one more failure, what does route_after_execution(state) return, and why?

Answer: "respond_error" — the first failure sets attempts=1 (1<2, retries); the second failure sets attempts=2 (2<2 is False, gives up). Two total execute_sql calls, one retry.

3. What's the concrete risk of schema.py and load_data.py's SCHEMAS dict drifting apart?

Answer: The LLM would confidently generate SQL referencing columns that don't exist (or miss ones that do), since generate_sql only ever sees schema.py's hand-written description, never the live database structure.

4. Why is check_sql()'s keyword blocklist described as biased toward false positives rather than false negatives — and what's a concrete false positive it can produce?

Answer: It's a token-level regex, not a SQL parser, so it can't distinguish a forbidden keyword used as SQL syntax from the same word appearing as string data — e.g. WHERE category = 'grant' gets rejected even though it's a harmless SELECT.

5. If you extended this system to handle a question returning 120 raw rows (not an aggregate), what would the Narrator actually see, and why might that matter?

Answer: Only the first 50 — execute_sql caps at 200 via fetchmany(200), then summarize further truncates to rows[:50] before prompting. The "Evidence" in the answer would be silently grounded in an unlabeled subset of the real result set.

13. References

- README.md — setup, architecture diagram, and design-decision notes
- CLAUDE.md — the condensed orientation doc for AI coding assistants working in this repo; largely mirrors this document at lower resolution
- ai_usage.md — the primary source for Section 8's bug catches, with full narrative detail
- src/agent/*.py — read in this order for the fastest ramp-up: state.py, schema.py, safety.py, db.py, nodes.py, graph.py